

AI4Math 与 AI4TCS 交叉应用讲义
从 generator-verifier 闭环到可审计研究项目

RUCmonk2

2026 年 8 月

如何使用这本讲义

本讲义用于把信息论、随机算法与 Lean 基础转成可执行的 AI4Math / AI4TCS 研究能力。它不以罗列热门模型为目标，而是反复追问六件事：研究对象是什么，怎样表示，谁生成候选，谁验证，能够给出什么保证，以及结果怎样跨实例或规模泛化。

阅读每章时维护一张证据登记表：

字段	必须写清的内容
Claim	想支持的最小、可证伪结论
Object / representation	搜索的对象及其编码是否保留结构
Generator	人、枚举、启发式、语言模型或学习器
Verifier	精确检查、形式证明、随机测试或代理指标
Guarantee	kernel-checked、概率保证、有限枚举或仅经验结果
Generalization	新实例、新分布、新规模和最坏情况分别如何
Audit artifact	代码、配置、反例、证明、日志和结果表

正文把三个建议项目扩写成完整方法，但仍遵守“只选一个两周闭环”的范围约束。完成一个有失败分析的小项目，比同时搭三个空框架更接近真实科研。

目录

如何使用这本讲义	1
第一章 领域边界、任务地图与证据等级	6
1.1 AI4Math、AI4TCS 与 TCS4AI	6
1.2 AI4Math 不是单一 benchmark	6
1.3 AI4TCS 的对象地图	7
1.4 统一的六字段分析	7
1.5 表示不是中性容器	7
1.6 Verifier 的证据光谱	8
1.7 三个常见结论偷换	8
1.8 从兴趣到可证伪问题	9
1.9 三个项目的定位	9
第二章 Generator–Verifier–Search 闭环	10
2.1 把闭环写成数学对象	10
2.2 Verifier 的 soundness 与 completeness	10
2.3 代理目标与真实性质	11
2.4 分层 verifier 漏斗	11
2.5 反馈的信息量与适应性过拟合	11
2.6 表示、规范化与等价类	12
2.7 反例比一个负分更有价值	12
2.8 一个最小搜索框架	12
2.9 何时停止	13
第三章 项目 A：信息论与学习增强 Sketch	14
3.1 问题定义必须先固定流模型	14
3.2 经典 Count-Min baseline	14
3.3 预测器到底输出什么	15

3.4	三种最小设计	15
3.4.1	预测 heavy hitters 分流	15
3.4.2	对 residual 做 sketch	15
3.4.3	按预测负载分配宽度	16
3.5	Prediction error 不能只选一个数字	16
3.6	Consistency 与 robustness	16
3.7	信息论视角怎样使用	17
3.8	实验数据与变量	17
3.9	Verifier 与输出表	18
3.10	Mergeability 检查	18
3.11	两周停止条件	18
第四章	项目 B: 随机算法候选的自动验证器	20
4.1	先选一个可穷举的小对象	20
4.2	候选接口与纯函数边界	20
4.3	正确性与性能必须分层	21
4.4	五层 verifier	21
4.5	小规模穷举如何设计	21
4.6	Hidden tests 的生命周期	22
4.7	反例生成与缩小	22
4.8	Freivalds 作为分层验证例子	22
4.9	等价去重与原创性	23
4.10	复杂度不能由少量 timing 拟合	23
4.11	最小 generator	23
4.12	指标与失败分类	24
第五章	项目 C: Lean 自动形式化与随机算法证明	25
5.1	第一题应当小而完整	25
5.2	三列表保证 statement fidelity	25
5.3	先证明不含概率的结构 lemma	26
5.4	一个状态不变量起点	26
5.5	随机性形式化的两条路线	27
5.5.1	有限枚举路线	27
5.5.2	mathlib 概率/测度路线	27
5.6	Autoformalization 的输入不只有题目一句话	27

5.7	Proof search 与 statement 生成分离	27
5.8	Lean、SMT、穷举与随机测试分工	28
5.9	无占位验收	28
5.10	推荐项目结构	28
5.11	两周节奏	29
第六章	实验设计、反例与适应性过拟合	30
6.1	从最小 claim 反推实验	30
6.2	实验单位与独立性	30
6.3	Baseline 和预算公平性	30
6.4	数据划分与冻结顺序	31
6.5	报告分布, 不只报告平均值	31
6.6	零失败不是零概率	31
6.7	多次尝试与 winner's curse	32
6.8	分布外和对抗测试	32
6.9	Property-based testing 与 metamorphic relation	32
6.10	反例最小化协议	33
6.11	失败分类表	33
6.12	可复现运行清单	33
第七章	两周项目执行与可复现交付	35
7.1	先用决策表只选一个项目	35
7.2	第 0 天: 冻结项目契约	35
7.3	十个工作日节奏	36
7.4	目录模板	36
7.5	三个项目的阶段闸门	37
7.5.1	项目 A	37
7.5.2	项目 B	37
7.5.3	项目 C	37
7.6	证据登记表模板	37
7.7	研究笔记结构	37
7.8	Go / revise / stop	38
7.9	从两周项目到研究问题	38
7.10	发布前安全与仓库审计	38
第八章	综合习题与参考答案	40

8.1 习题	40
8.2 参考答案与提示	41

第一章 领域边界、任务地图与证据等级

1.1 AI4Math、AI4TCS 与 TCS4AI

三个相近名称对应不同方向：

AI4Math 用 AI 辅助数学问题表示、证明、形式化、猜想和研究协作；

AI4TCS 用 AI 辅助算法发现、复杂性探索、组合优化、程序综合、求解器与验证；

TCS4AI 用理论计算机科学分析学习系统的泛化、优化、鲁棒性、隐私、复杂度和可验证性。

一个项目可能同时涉及多项，例如用学习器生成算法候选属于 AI4TCS，而分析该学习器能查询 verifier 多少次、会泄漏多少信息属于 TCS4AI。必须先说明当前论文或实验在回答哪一个方向的问题。

1.2 AI4Math 不是单一 benchmark

至少区分五类对象：

1. **自然语言解题**：输出可读推理或答案，常由标准答案、规则或人工评分；
2. **形式定理证明**：statement 固定，输出 proof term/tactic，由 proof assistant 检查；
3. **自动形式化**：把自然语言和上下文翻译为定义与 statement；
4. **猜想或对象发现**：生成新的序列、构造、恒等式、反例或几何辅助对象；
5. **研究级协作**：检索文献、运行计算、提出假设、验证局部 lemma 并维护证据链。

形式证明通过率主要评估第二类；它不能直接替代 statement 翻译忠实度，也不能说明生成结论新颖或重要。

1.3 AI4TCS 的对象地图

方向	候选对象	核心 verifier
算法发现	程序、递推式、更新规则	正确性测试、形式证明、成本模型
组合优化	可行解、局部搜索策略	约束检查、目标值、近似界
矩阵乘法	双线性分解、张量分解	代数恒等式、乘法次数、数值稳定性
程序综合	满足规范的程序	类型、测试、SMT/证明、资源限制
求解器设计	分支、变量、重启启发式	求解正确性与实例分布性能
复杂性探索	上界构造、下界证据	一般性证明、归约、反例与文献审查
数据流/Sketch	状态更新、估计器、参数策略	误差、失败概率、空间与 mergeability

同一个 evaluator 分数可能混合正确性和性能。研究设计应先让错误候选得不到“性能优秀”的奖励，再讨论如何在正确集合中优化成本。

1.4 统一的六字段分析

对任何代表工作或自己的想法，固定填写：

Object 搜索的是 proof、program、公式、张量分解还是实验配置；

Representation token、语法树、DSL、图、张量或 Lean expression；

Generator 枚举、进化、强化学习、语言模型、人类启发式或混合系统；

Verifier 精确、形式、随机、经验或代理评价；

Guarantee 一般定理、有限范围证明、高概率界或经验改进；

Generalization 是否跨实例、分布、规模、表示和硬件。

这六字段可以防止把“模型更强”“验证过了”“算法更快”等模糊句子直接当结论。

1.5 表示不是中性容器

设真实对象集合为 $\mathcal{O}$ ，编码空间为 $\mathcal{Z}$ ，解码器 $d: \mathcal{Z} \rightarrow \mathcal{O}$ 。表示会决定：

- 哪些对象可表达，哪些被排除；
- 等价对象是否出现大量重复编码；

- 局部编辑是否对应有意义的对象变化；
- verifier 是否容易计算；
- 生成器是否能利用对称性、类型和不变量。

例如直接生成 Python 字符串搜索空间巨大且包含大量语法错误；在类型化 DSL 中生成可消除解析错误，但 DSL 可能排除真正有价值的算法。因此“约束表示”同时带来搜索效率和表达偏差。

1.6 Verifier 的证据光谱

证据	可以可靠支持	单独不能支持
proof assistant kernel	固定形式 statement 可证明	自然语言翻译、原创性、工程性能
SAT/SMT unsat	编码公式在求解域中无反例	编码忠实、域外或无限规模结论
小规模穷举	枚举范围内正确/存在反例	任意规模正确
随机测试	指定分布和预算下经验表现	worst-case guarantee
精确数值 evaluator	当前实例目标值	跨实例/跨规模泛化
人工同行审查	意义、文献和论证质量	机械无误或完全可复现

“自动 verifier”并不自动等于“形式 verifier”。它也可能只是运行一组测试、计算一个代理分数或调用数值模拟。

1.7 三个常见结论偷换

固定实例加速不等于渐近复杂度下降。 若在固定矩阵尺寸上减少标量乘法次数，得到的是该尺寸构造的改进。要声称矩阵乘法指数改进，还需一族可推广构造及其渐近分析。

benchmark 最优不等于发现新算法。 候选可能与已知算法等价，只是变量重命名、操作重排或针对测试集调参。需要规范化、等价性检查和文献检索。

kernel 接受不等于原题解决。 若 statement 漏掉条件、量词错位或结论被改弱，证明完全正确仍可能没有回答原题。

结论边界

任何“首次”“更优复杂度”“已证明”“自动发现”的表述都应附上对象、比较基线、验证器、适用范围和证据来源。若这些字段写不出来，应先缩小 claim，而不是补强语气。

1.8 从兴趣到可证伪问题

“我想做 AI4Math” 不是研究问题。可执行问题应包含对象、变量、对照和失败条件，例如：

在固定内存预算下，使用上一窗口频率预测重分配 Count-Min 宽度，能否在 Zipf 与分布漂移流上降低重尾 key 的 95% 分位绝对误差，同时在错误预测时不劣于经典 baseline 超过预设比例？

这个问题明确了对象、预测接口、指标、数据分布、baseline 和 robustness 阈值；即使答案与否，也能产生反例和后续判断。

1.9 三个项目的定位

项目	训练的核心能力	最终可信边界
A 学习增强 Sketch	prediction interface、误差与鲁棒实验	exact counter 对照 + 明确理论/经验边界
B 算法候选验证器	多层 verifier、反例、隐藏测试	有限正确性/经验泛化，不冒充复杂度证明
C Lean 形式化	statement fidelity、lemma 分解、kernel proof	固定 statement 的形式正确性

三者共享 generator-verifier 思维，但交付不相同。一次只做一个，才能把失败分析做实。

本章验收

1. 用自己的话区分 AI4Math、AI4TCS 与 TCS4AI；
2. 对一个代表工作填写六字段分析，不使用“模型更强”作答案；
3. 举例说明表示如何改变可搜索对象和等价重复；
4. 为 kernel、SMT、穷举和随机测试分别写出证据边界；
5. 把一个宽泛兴趣改写成含 baseline、指标和失败条件的问题。

第二章 Generator–Verifier–Search 闭环

2.1 把闭环写成数学对象

设候选空间为 $\mathcal{C}$ ，问题实例为 x ，生成器在历史 h_t 条件下提出

$$c_t \sim G(\cdot \mid x, h_t).$$

Verifier 返回反馈

$$y_t = V(x, c_t),$$

搜索器再更新历史 $h_{t+1} = (h_t, c_t, y_t)$ 。终止后输出候选 $\hat{c}$ 和证据 e 。这个抽象涵盖 proof search、程序综合、组合构造、算法调参与反例引导修复。

必须额外定义：候选是否合法、反馈含哪些字段、查询预算、终止条件、最终候选是否由比搜索阶段更强的 verifier 复验。

2.2 Verifier 的 soundness 与 completeness

若真实目标性质为 $P(c)$ ，二值 verifier 返回接受/拒绝。理想 soundness 是

$$V(c) = \text{accept} \Rightarrow P(c),$$

即不接受错误候选；completeness 是

$$P(c) \Rightarrow V(c) = \text{accept},$$

即不拒绝正确候选。

形式 kernel 对固定 statement 的检查接近 sound verifier；有限测试通常两者都不完美：可能漏掉隐藏错误，也可能因超时拒绝正确但慢的程序。实验必须记录 false positive、false negative 与 unknown/timeout，而不是只有一个通过率。

2.3 代理目标与真实性质

昂贵性质 P 常被廉价分数 $S(c)$ 代理。例如用随机测试正确率代理全输入正确性，用小规模运行时间代理渐近性能，用 proof length 代理证明可读性。代理有用的条件是它与最终目标的关系可分析或至少在保留集上验证。

结论边界

生成器会优化你实际给出的分数，而不是你心里想要的性质。只要代理存在漏洞，足够适应性的搜索就可能找到高分但无意义的候选。Verifier 设计不是搜索之后的验收步骤，而是问题定义的一部分。

2.4 分层 verifier 漏斗

成本从低到高安排检查：



廉价层尽早拒绝明显错误，昂贵层只处理少量候选。每一层都要返回结构化失败原因，否则搜索器只能盲目重采样。

2.5 反馈的信息量与适应性过拟合

若每次反馈最多 b bit， T 轮 transcript 的熵至多 Tb bit，因此候选可从 verifier 获得的信息也受此上界约束。这个观察不能直接给出泛化定理，却提醒我们：分数、小数精度、错误位置和完整反例都在泄漏测试信息。

当生成器反复根据同一 validation 集反馈修改候选时，validation 已参与训练。即使没有显式梯度，也会发生适应性过拟合。对策包括：

- 限制查询次数和反馈精度；
- 搜索、validation、hidden、final audit 使用不同数据；

- hidden 结果只在固定里程碑揭示；
- 最终候选在新生成实例或独立实现上复验；
- 完整记录每次查询，不能只展示最优一次。

2.6 表示、规范化与等价类

若候选 c 与 c' 只差变量重命名、操作交换或图同构，搜索器重复探索它们会浪费预算，也会夸大候选多样性。定义等价关系 $c \sim c'$ 后，应尽可能使用规范形 $\text{canon}(c)$ 或 hash 去重。

但规范化本身需要验证：

$$c \sim c' \Rightarrow P(c) = P(c'), \quad S(c) = S(c')$$

是否成立取决于目标。两个代数表达式功能等价，数值稳定性和硬件性能仍可能不同，因此不能在所有评价层共用同一等价关系。

2.7 反例比一个负分更有价值

设候选在输入 x 上失败。理想反馈包含：

1. 可重放的输入与 seed；
2. 期望输出、实际输出和失败谓词；
3. 失败层级（解析、正确性、性能、超时等）；
4. 缩小后的最小反例；
5. 当前 verifier 版本与配置 hash。

反例缩小可以删除输入元素、减小数值、收缩图或简化表达式，只要失败仍保留。最小反例既帮助人理解，也给生成器更清晰的修复信号。

2.8 一个最小搜索框架

```
history = []
best = None

for step in range(search_budget):
    candidate = generator.propose(problem, history)
    candidate = canonicalize(candidate)
```

```

result = cheap_verifier(candidate)
if result.passed:
    result = validation_verifier(candidate)

history.append((candidate, result.summary))
best = update_best(best, candidate, result)

final_result = hidden_and_exact_audit(best)
save_run(problem, history, best, final_result)

```

框架故意把搜索反馈与最终审计分开。`best` 不能仅按性能选；首先必须满足明确正确性门槛。

2.9 何时停止

预先定义停止条件：查询预算耗尽、连续若干轮无新等价类、正确候选已达到目标、verifier 漏洞被发现、运行资源超过上限，或核心 claim 已被反例否定。停止并不等于失败；明确的否定结果可以防止继续堆模型复杂度。

可执行协议

每次运行固定保存：problem version、候选 grammar、生成器版本、所有 seed、查询预算、每层 verifier 配置、完整候选/反馈轨迹、最小反例、最终独立审计和 wall-clock/资源统计。

本章验收

1. 写出自己项目的 $\mathcal{C}, G, V, h_t$ 和终止条件；
2. 区分 verifier soundness、completeness、timeout 与 unknown；
3. 设计一个从语法检查到最终形式审计的成本漏斗；
4. 列出反馈中会泄漏 validation 信息的字段；
5. 定义候选等价关系和规范化边界；
6. 把一个失败输入缩成更小反例并保存可重放信息。

第三章 项目 A：信息论与学习增强

Sketch

3.1 问题定义必须先固定流模型

设 universe 为 $[N]$ ，频率向量 $f \in \mathbb{R}^N$ 由更新 (i, Δ) 隐式定义。至少区分：

insertion-only $\Delta \geq 0$ ，频率只增加；

strict turnstile 允许负更新，但任意时刻 $f_i \geq 0$ ；

general turnstile 中间和最终频率都可能为负。

Count-Min 的单侧估计依赖非负碰撞噪声，不能不加说明地迁移到 general turnstile。项目第一行就应声明模型、查询类型、passes 和 adversary 是否看到 hash seed。

3.2 经典 Count-Min baseline

选择 d 个相互独立的通用哈希函数 $h_j : [N] \rightarrow [w]$ ，维护计数器

$$C_{j, h_j(i)} \leftarrow C_{j, h_j(i)} + \Delta.$$

点查询返回

$$\hat{f}_i = \min_{j \in [d]} C_{j, h_j(i)}.$$

在 insertion-only 模型下始终有 $\hat{f}_i \geq f_i$ 。标准参数量级

$$w = \Theta(1/\varepsilon), \quad d = \Theta(\log(1/\delta))$$

给出对固定查询 i 的高概率加性界

$$\mathbb{P}[\hat{f}_i > f_i + \varepsilon \|f\|_1] \leq \delta,$$

常数取决于具体参数化。学习增强方法必须与这个明确 baseline 比较，而不是只和 exact counter 比。

3.3 预测器到底输出什么

“有一个 prediction” 不够。常见接口包括：

1. 每个 key 的频率预测 $p_i \approx f_i$;
2. 归一化分布预测 $q_i \approx f_i / \|f\|_1$;
3. heavy-hitter 集合预测 $H \subseteq [N]$;
4. 上一窗口状态或分桶负载预测;
5. 查询分布预测，而不是数据频率预测。

这些接口改变的目标不同。频率预测可能用于 sketch residual; heavy-hitter 预测用于分流; 查询分布用于把资源给常查 key。必须写清预测在流开始前给定、在线更新，还是会看到部分真实流。

3.4 三种最小设计

3.4.1 预测 heavy hitters 分流

对预测集合 H 使用 exact counters, 其余 key 进入 sketch。若错过一个真实 heavy hitter, 它仍会污染 sketch。为了保留 fallback, 可以让所有 key 同时进入 backup sketch:

```
def update(item, delta):
    backup_sketch.update(item, delta)
    if item in predicted_heavy:
        exact[item] += delta

def query(item):
    if item in predicted_heavy:
        return exact[item]
    return backup_sketch.query(item)
```

这在固定 backup 参数下不会比 baseline 更差, 却额外使用 exact counter 内存。若总内存固定, 就必须缩小 backup sketch, 此时 robustness 需要重新测量或证明。

3.4.2 对 residual 做 sketch

维护预测 p 与 residual $r = f - p$, 查询时返回

$$\hat{f}_i = p_i + \hat{r}_i.$$

即使 $f_i, p_i \geq 0$, r_i 也可能为负, 因此不能直接沿用 Count-Min 的非负噪声证明。可考虑 Count Sketch、正负 residual 分开或带显式截断的估计器; 每种选择都改变偏差与方差。

若 residual 范数 $\|f - p\|_1$ 明显小于 $\|f\|_1$, 理论上有机会把误差尺度从全频率质量换成 residual 质量。但这需要严格证明所用 sketch 在当前更新模型上成立。

3.4.3 按预测负载分配宽度

预测分桶负载或 key 重要性后, 给高风险区域更多计数器。优点是仍保留哈希结构; 风险是自适应分配可能破坏原哈希独立性、mergeability 或 worst-case 界。最小实验应先把分配规则冻结, 再生成流, 避免使用测试流真值调宽度。

3.5 Prediction error 不能只选一个数字

若预测和真实频率都可归一化为分布 p, q , 可考虑:

$$\|p - q\|_1, \quad \text{TV}(p, q) = \frac{1}{2}\|p - q\|_1, \quad D_{\text{KL}}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i}.$$

KL 对 $q_i = 0 < p_i$ 极敏感, 需要平滑与方向说明。若预测目标是 heavy hitters, precision/recall、漏掉的频率质量

$$\sum_{i \in H^* \setminus H} f_i$$

往往比全分布 KL 更直接。若关心查询加权误差, 还要引入查询分布。

结论边界

先观察哪种误差指标与收益相关, 再事后把它命名为 prediction error, 会产生选择偏差。项目开始前应指定主误差指标和一个辅助指标; 其他相关性只作为探索性结果。

3.6 Consistency 与 robustness

设经典 baseline 在预算 M 下的损失为 $L_B(f; M)$, 学习增强算法损失为 $L_A(f, p; M)$, 预测误差为 $\eta(f, p)$ 。一种项目级定义是:

consistency 当 $\eta = 0$ 或很小时, L_A 显著优于 L_B ;

robustness 对任意 p , $L_A(f, p; M) \leq \beta L_B(f; M) + \gamma$;

smoothness 损失随 η 逐渐恶化, 而不是在小误差处突然崩溃。

这里 L 、 β 、 γ 和预算必须具体化。若只能实验测得, 就应写“经验 robustness”, 不能称为一般性保证。

一个理想形式可能是

$$L_A(f, p; M) \leq \min\{g(\eta(f, p), M), \beta L_B(f; M) + \gamma\},$$

但能否证明取决于设计, 不应先把目标公式当成 theorem。

3.7 信息论视角怎样使用

预测 P 可视为关于真实频率对象 F 的 side information。互信息

$$I(F; P) = H(F) - H(F | P)$$

衡量平均不确定性减少; 条件熵低意味着理想编码中描述 F 所需额外信息更少。这提供研究语言, 却不能直接推出某个 sketch 的空间上界。

要把信息量转成算法 theorem, 还需:

1. 明确 F, P 的随机模型与分布;
2. 指定算法能怎样访问 P ;
3. 构造编码/通信协议或数据结构;
4. 证明恢复误差、失败概率和计算成本;
5. 给出 converse 或 baseline 才能讨论最优性。

3.8 实验数据与变量

最小数据生成器覆盖:

- 均匀频率, 检查预测机制是否无谓增加开销;
- Zipf 重尾, 观察 heavy hitter 分流;
- burst, 检查短期尖峰;
- distribution drift, 预测来自上一窗口;
- adversarial miss, 把最大 key 故意排除出预测集合;
- general-turnstile toy, 验证算法是否越过模型边界。

控制变量至少包括 w, d 、总内存、流长度、universe、seed、预测误差强度和查询集合。所有方法必须使用同一计费口径: 计数器位宽、预测存储、hash seed 和 metadata 都应计入空间。

3.9 Verifier 与输出表

Exact counter 给真值。对每个 key 记录

$$e_i^{\text{abs}} = |\hat{f}_i - f_i|, \quad e_i^{\text{rel}} = \frac{|\hat{f}_i - f_i|}{\max(1, |f_i|)}.$$

不要只报平均值；至少报告 median、95%/99% 分位、最大误差、heavy hitters 子集误差、空间和更新时间。

method	stream	pred. error	memory	p95 abs.	max abs.
baseline	zipf	—	固定	数值	数值
augmented	zipf	数值	固定	数值	数值

每行还应由 run id 关联 seed、配置和原始结果；表中聚合值不能替代逐次运行记录。

3.10 Mergeability 检查

经典线性 sketch 常可合并：分别处理两个子流的状态相加，等于处理并流。加入预测后要问：两个分片是否使用同一预测、同一 heavy set 和同一哈希？预测模型参数是否属于状态？若各分片依据本地分布自适应分配内存，合并后的状态可能不再等价于全局运行。

3.11 两周停止条件

第 1 周完成 exact + Count-Min、数据生成器、固定内存计费和无预测基线；第 2 周只实现一种 prediction interface，做错误预测与漂移测试。出现以下任一条件就停止加模型：无法定义 prediction error、固定预算下没有收益、收益只来自泄漏真值、backup 被不公平缩小、或 mergeability/流模型已改变却没有声明。

可执行协议

最低交付：一个无 GPU Python baseline、固定 schema 的 `summary.csv`、主误差曲线、空间计费表、至少一个“预测有害”的最小反例，以及一页说明哪些结论是经典 theorem、哪些只是本实验观察。

本章验收

1. 写清流模型、查询类型和 Count-Min baseline 保证；
2. 为频率、分布、heavy set 三种预测接口分别定义误差；
3. 解释 residual 为何可能破坏 Count-Min 单侧分析；

4. 在固定总内存下定义 consistency、robustness 和 smoothness;
5. 设计包含 drift 与 adversarial miss 的实验矩阵;
6. 检查加入预测后 mergeability 是否仍成立。

第四章 项目 B：随机算法候选的自动验证器

4.1 先选一个可穷举的小对象

两周项目不训练大型 agent，目标是完整体验候选生成、分层验证、反馈、过拟合和最终审计。对象应满足：接口短、小规模有精确 oracle、能生成反例、性能指标可单独测量。

合适起点包括：

- 小型集合覆盖的选择顺序；
- 图遍历中的邻居排序或 tie-breaking 规则；
- 小规模调度/装箱启发式；
- 数据结构操作序列；
- 固定尺寸代数表达式或矩阵乘法候选；
- Count-Min 参数/查询组合的受限 DSL。

不要选择“任意 Python 程序解决 NP-hard 问题”这类无限且无精确边界的空间。

4.2 候选接口与纯函数边界

候选应是可隔离调用的纯函数或受限 DSL：

```
def candidate(instance, config):  
    """Return a solution object, not a printed explanation."""  
    ...
```

Verifier 负责检查返回对象，不信任候选自己报告的分值。必须限制运行时间、内存、文件/网络访问和输出大小；即使项目完全由自己生成，也要把候选当作不可信代码处理。

更安全的入门方案是让 generator 输出 JSON/AST，由自己的解释器执行，而不是直接 eval 任意字符串。类型化 grammar 可在生成阶段排除大量语法与接口错误。

4.3 正确性与性能必须分层

设可行性谓词为 $F(x, c)$ ，目标函数为 $J(x, c)$ 。候选先满足

$$F(x, c) = \text{true},$$

再在可行集中比较 J 。评分可用字典序：

$$\text{score}(c) = (\mathbf{1}[\text{all mandatory correctness tests pass}], -J(c)).$$

这样“输出空解、速度极快”不会胜过正确但稍慢的算法。若允许近似解，需把可行性和 approximation ratio 分开。

4.4 五层 verifier

1. **接口层**：解析、类型、shape、数值范围、超时与资源上限；
2. **已知单测层**：手工边界例、历史反例和退化输入；
3. **小规模精确层**：枚举所有输入或用可信 solver/oracle；
4. **随机/OOD 层**：不同分布、规模和 seed，保存第一个失败；
5. **最终审计层**：形式不变量、SMT、独立实现、复杂度与文献检查。

每层返回结构化状态 pass/fail/timeout/unknown 和证据。unknown 不能按 pass 处理。

4.5 小规模穷举如何设计

若输入空间在规模 $n \leq n_0$ 时有限，枚举器应做到：

- 每个合法实例恰好被覆盖或可证明覆盖；
- 同构去重不会误删语义不同实例；
- oracle 独立于候选实现；
- 失败时保存最小规模和字典序最小反例；
- 枚举边界 n_0 、实例数量和耗时进入报告。

“通过所有 $n \leq 8$ 图”是有限 theorem-like 证据，但仍不是任意 n 正确。它可以帮助猜不变量和构造形式证明。

4.6 Hidden tests 的生命周期

建议四份数据:

development 公开边界例, 可反复调试;

validation 搜索器可得到有限反馈, 用于选择候选;

hidden 搜索过程中不可读取, 只在固定 checkpoint 评测;

OOD/final 新分布、新规模或新生成器, 在候选冻结后一次审计。

如果每轮都返回 hidden 的详细错误, 它就变成 validation。项目应记录查询次数和揭示的反馈; “文件名叫 hidden” 并不能提供隔离。

4.7 反例生成与缩小

反例可能来自穷举、随机/property-based 测试、SMT 或人工边界分析。找到 x 后, 定义保持失败的谓词

$$\text{fails}(x, c).$$

缩小操作按对象设计: 删除图的边/点、缩短操作序列、减小整数、删除集合元素、降低矩阵维数。使用 delta debugging 时每次只接受仍满足 `fails` 的更小对象。

一个反例记录应包含: 序列化输入、candidate hash、seed、oracle 输出、候选输出、失败谓词、verifier version 和复现命令。

4.8 Freivalds 作为分层验证例子

验证矩阵候选 $C = AB$ 时, 直接乘法成本可能高。Freivalds 在域上随机取向量 $r \in \{0, 1\}^n$, 检查

$$A(Br) = Cr.$$

若 $AB = C$, 总是接受; 若不等, 单次错误接受概率至多 $1/2$ 。独立重复 k 次后至多 2^{-k} 。

分层协议可为:

1. shape/dtype 检查;
2. 小尺寸直接计算 AB ;
3. 大尺寸多次 Freivalds 快速筛选;
4. 最终候选做精确乘法、符号恒等式或分解等式验证;
5. 运行时间、乘法次数、数值稳定性另行报告。

结论边界

实现必须说明运算域、整数溢出和随机源。浮点近似相等不满足上述精确域证明；重复使用同一向量也不能按独立重复计算 2^{-k} 。

4.9 等价去重与原创性

设候选是操作序列。可先消除死代码、常量折叠、重命名临时变量，再 hash 规范形。对代数构造还可能有置换、缩放或基变换等对称。

但“规范形不同”仍不等于原创。最终候选需要：

1. 与已知 baseline 做功能和成本对照；
2. 搜索相关论文、代码和 benchmark 提交；
3. 检查是否只是参数、硬件或固定实例特化；
4. 明确新意在构造、搜索方法、理论还是工程实现。

4.10 复杂度不能由少量 timing 拟合

对规模 $n = 4, 8, 16$ 的运行时间拟合直线或幂律，只能形成经验假设。渐近结论需要分析操作数、递归式、数据结构和最坏/平均输入。固定预算 benchmark 改进可以如实表述为“在给定实例分布和硬件上更快”。

候选若包含预处理、模型调用或巨大常数，也必须计入端到端成本。不要只统计核心函数中最有利的一段。

4.11 最小 generator

Generator 不必一开始就是大模型。可按顺序实现：

1. 随机采样合法 grammar；
2. 对当前候选做局部 mutation；
3. 根据失败类型选择 mutation；
4. 最后才接语言模型提出候选或修复。

若随机/枚举 baseline 已能找到相同结果，就能更准确衡量语言模型的增量价值。

4.12 指标与失败分类

报告：合法候选率、mandatory tests 通过率、hidden/OOD 通过率、首次正确候选所需查询数、唯一等价类数、最小反例规模、wall-clock、timeout 率和最佳可行目标值。

失败分类至少包括 parse/type、exception、timeout、不可行、功能错误、数值问题、过拟合 validation、重复候选和 verifier 漏洞。

可执行协议

两周最低交付：一个受限候选接口、一个独立 oracle、development/validation/hidden/OOD 四层实例生成器、一张失败分类表、至少一个缩小反例，以及一份说明哪些性质只经验验证、哪些已有形式或一般性保证的笔记。

本章验收

1. 选择一个可精确枚举的小对象并定义 candidate 接口；
2. 把正确性门槛和性能目标写成两个函数；
3. 设计五层 verifier 和每层的 timeout/unknown 行为；
4. 定义 hidden 数据何时可被查询以及反馈精度；
5. 为对象写三个反例缩小操作；
6. 说明固定实例速度、经验 scaling 与渐近复杂度的区别。

第五章 项目 C: Lean 自动形式化与随机算法证明

5.1 第一题应当小而完整

目标不是复现大型 theorem-proving benchmark, 而是经历完整链路: 自然语言 $\rightarrow$ 数学形式 $\rightarrow$ Lean statement $\rightarrow$ lemma 依赖 $\rightarrow$ kernel proof $\rightarrow$ 语义审查。

推荐梯度:

1. 列表/状态的确定性大小不变量;
2. 有限集合上 indicator 的和等于事件基数;
3. 非空有限均匀样本空间概率总和为 1;
4. 两个或有限多个事件的 union bound;
5. 简化抽样估计器的无偏性;
6. 最后才挑战 reservoir sampling 分布或一般集中不等式。

完整 Chernoff bound、连续分布和端到端 Count-Min theorem 需要更成熟的概率库经验, 不适合作为第一条正式交付。

5.2 三列表保证 statement fidelity

以有限 union bound 为例:

层	内容
自然语言	在同一有限均匀样本空间中, 若干事件并集的概率不超过各事件概率之和
数学式	$\Pr[\bigcup_{i \in I} E_i] \leq \sum_{i \in I} \Pr[E_i]$, Ω 有限非空
Lean 设计	选择 Finset/Set 表示事件族, 定义均匀概率或复用概率库, 显式给出有限性、可判定性与非空假设

对齐时逐项检查: 同一个样本空间、索引集有限、概率分母不为零、事件表示支持并

集、数值域可比较。若 Lean statement 只证明两个事件，就不能在标题中写“任意事件族”。

5.3 先证明不含概率的结构 lemma

Union bound 的核心可先写成集合计数：

$$\left| \bigcup_i E_i \right| \leq \sum_i |E_i|.$$

然后再处理除法与概率包装。类似地，抽样算法先证明状态大小、索引与逐步转换，再进入分布。

这种分解有三点好处：

1. 更容易在 mathlib 中找到已有 Finset/Set lemma；
2. 失败时能区分组合结构、代数归约和概率接口；
3. 未来替换均匀概率为一般测度时可复用结构层。

5.4 一个状态不变量起点

```
import Mathlib

structure SampleState where
  seen : Nat
  kept : List Nat

def SampleState.Inv (k : Nat) (s : SampleState) : Prop :=
  s.kept.length <= k

def SampleState.empty : SampleState :=
  { seen := 0, kept := [] }

theorem SampleState.empty_inv (k : Nat) :
  SampleState.Inv k SampleState.empty := by
  simp [SampleState.Inv, SampleState.empty]
```

这条 theorem 很简单，但接口完整：状态、容量参数、不变量、初始状态和证明都明确。下一步可定义一个只在未满足时插入的转换，并证明保持 Inv；再进一步才加入随机替换位置。

5.5 随机性形式化的两条路线

5.5.1 有限枚举路线

把 random tape 定义为有限类型，概率用满足事件的 tape 数量除以总数。优点是直观、可执行、适合小算法；缺点是重复建立概率代数，扩展到复杂独立性或分布较笨重。

5.5.2 mathlib 概率/测度路线

复用一般 measure、probability measure、随机变量与期望理论。优点是能连接成熟 theorem；缺点是类型类、可测性和积分接口学习成本高。初学项目可以先完成有限版本，再写一页“迁移到一般概率库还缺什么”。

不要在同一 theorem 中混用自定义概率和 mathlib probability notation 而不写转换 lemma。

5.6 Autoformalization 的输入不只有题目一句话

高质量自动形式化输入应包含：

- 术语定义和上下文；
- 预期类型与数学域；
- 可使用的 mathlib 概念；
- 正例、边界例与反例；
- 不允许补加的假设；
- statement 对齐标准。

让模型自由“补齐合理条件”可能得到可证明但错误的 statement。更稳妥的流程是生成多个候选 statement，先用类型检查和有限例测试，再由人逐项对齐量词与假设。

5.7 Proof search 与 statement 生成分离

先冻结 statement S ，再搜索 proof $p : S$ 。若 proof 搜索失败，不应默认修改 S ；应分类：缺定义、缺 lemma、归纳结构不对、库 API 不熟或 statement 实际为假。

若确实需要改 statement，创建新版本并记录差异：

字段	记录
Old statement	原量词、假设与结论
New statement	修改后的完整声明
Reason	修复错误、缩小第一版范围或加入必要条件
Semantic effect	更强、更弱、不同域或不同对象
Validation	反例、人工对齐或已有数学来源

5.8 Lean、SMT、穷举与随机测试分工

Lean 组合抽象定义、归纳证明和库 theorem，提供 kernel-checked 证据；

SMT/SAT 快速处理有限约束、位向量、线性算术或找模型；

穷举 对小有限域建立强 oracle 并发现最小反例；

随机测试 扩展到大实例，筛查实现错误和分布外行为；

Python reference 生成数据、统计实验和与 Lean 可执行定义做 differential testing。

工具应形成证据梯度。随机测试发现反例可以推翻 theorem 候选；测试没发现反例不能代替 proof。

5.9 无占位验收

最终构建除了退出码为零，还应审计：

- 是否出现 sorry、admit 或新增 axiom；
- theorem 是否仍是冻结的 statement；
- imports 是否意外引入同名 theorem 直接循环使用；
- 是否依赖非预期 native/oracle 路径；
- toolchain、manifest 和构建命令是否提交；
- #print axioms theoremName 在当前版本显示什么依赖。

使用经典选择等标准公理不一定是问题，但必须区分“依赖哪些公理”和“偷偷加入一个恰好等于目标的 axiom”。

5.10 推荐项目结构

```
lean-randomized-theorem/
  README.md
```

```

Problem.md
ProofStatus.md
lean-toolchain
lakefile.toml
LeanRandom/
  Definitions.lean
  FiniteProbability.lean
  MainTheorem.lean
  Tests.lean
scripts/
  enumerate.py

```

`Problem.md` 对齐自然语言、数学与 Lean; `ProofStatus.md` 分已证明、仅测试、未形式化和外部假设。源码中的 `theorem` 不承担项目全部叙述。

5.11 两周节奏

第 1–2 天冻结题目和三列表; 第 3–4 天完成有限 Python 枚举; 第 5–7 天写 Definitions 与结构 lemma; 第 8–10 天证明主 theorem; 第 11 天生成变体和反例; 第 12 天无占位审计; 第 13–14 天整理失败轨迹与语义边界。

如果第 7 天仍卡在一般概率库接口, 应主动退回有限均匀版本, 而不是用 `sorry` 保留过大目标。

可执行协议

最低交付: 可 `lake build` 的固定版本项目、三列表 `Problem.md`、无占位核心 `theorem`、有限枚举脚本、至少一个失败/反例记录, 以及明确列出尚未形式化的概率、复杂度或实现层性质。

本章验收

1. 选择一条第一周可完成的有限 `theorem`;
2. 制作自然语言/数学/Lean 三列表并审查量词;
3. 把主 `theorem` 拆成结构、代数和概率三层 lemma;
4. 选择有限枚举或一般概率库路线并说明代价;
5. 冻结 `statement` 后设计 `proof search/repair` 协议;
6. 运行无 `sorry`/新增强公理的干净构建审计。

第六章 实验设计、反例与适应性过拟合

6.1 从最小 claim 反推实验

实验开始前写一句可证伪 claim:

在固定总内存和指定 Zipf/drift 数据生成器上，方法 A 相比 baseline B 降低重频 key 的 95% 分位绝对误差；当预测误差达到预设上限时，退化不超过阈值 τ 。

这句话决定需要控制内存、数据分布、key 子集、误差分位数、prediction error 和 robustness 阈值。若实验矩阵中没有这些变量，就无法支持 claim。

把结论分为：

confirmatory 预先指定主指标、方向和比较，回答核心问题；

exploratory 观察额外模式、相关性和意外失败，产生新假设。

探索结果可以报告，但不能事后伪装成预注册主结论。

6.2 实验单位与独立性

“跑了 100 次”是否有意义取决于独立单位。若 100 个 seed 共用同一条流，只改变 hash，估计的是算法随机性；若同时重采样流，混合了数据与算法随机性。应分层记录：

data seed, algorithm seed, generator/model seed, split seed.

重复使用同一模型 checkpoint 和同一 validation 反馈也会产生相关性，不能简单当 100 个独立样本。

6.3 Baseline 和预算公平性

至少包括：

1. exact oracle, 只用于真值和小规模成本参照；

2. 经典算法 baseline;
3. 无反馈/随机生成器 baseline;
4. 消融: 去掉预测、反馈、某层 verifier 或 fallback;
5. 简单启发式, 防止复杂模型只学到容易规则。

预算必须同口径: 内存包含参数与 metadata, 搜索成本包含所有 verifier 查询, 运行时间包含预处理与模型调用, 证明成本至少记录 wall-clock、搜索步数和环境规模。

6.4 数据划分与冻结顺序

推荐顺序: 先生成 development/validation 并公开规则; 冻结候选空间、主指标和预算; 再生成或封存 hidden/OOD; 搜索结束冻结候选; 最后只进行预定次数的审计。

若 hidden 失败后继续修改候选, 旧 hidden 已转化为训练信息。可以创建新 final set, 但必须在日志中写明迭代轮次, 不能仍称第一次 hidden 结果。

6.5 报告分布, 不只报告平均值

对独立观测 $X_1, \dots, X_m$, 样本均值与标准误差为

$$\bar{X} = \frac{1}{m} \sum_{j=1}^m X_j, \quad \text{SE}(\bar{X}) = \frac{s}{\sqrt{m}}.$$

但 sketch 最大误差、搜索步数和运行时间常重尾或受 timeout 截断, 均值不足。至少同时报告 median、四分位数、95%/99% 分位、最大值、失败率和有效样本数。

置信区间方法应匹配实验单位, 可使用成对比较、bootstrap 或适当参数方法。若方法共享同一输入实例, 优先报告 paired difference, 而不是把两组当独立样本。

6.6 零失败不是零概率

固定候选在 m 次独立 Bernoulli 测试中零失败, 只说明样本中未观察到失败。常用“rule of three”给出约 95% 上置信量级 $3/m$, 不是证明真实失败概率为 0。

若想以 Hoeffding 控制经验失败率 $\hat{p}$ 与真实分布失败率 p , 固定候选和独立同分布假设下有

$$\mathbb{P}[|\hat{p} - p| \geq \varepsilon] \leq 2e^{-2m\varepsilon^2}.$$

因此

$$m \geq \frac{\log(2/\delta)}{2\varepsilon^2}$$

足以给出概率至少 $1 - \delta$ 的加性估计。若候选是看过这些测试后选出的，不能直接套用同一固定候选界。

6.7 多次尝试与 winner's curse

从 K 个候选中选验证分数最高者，即使每个候选真实性能相同，最大经验分数也会偏高。搜索轮数越多、反馈越细，偏差越严重。

必须报告总候选数、有效等价类数、选择规则和最终独立复验。只展示“最佳 seed”相当于隐藏多重比较。模型超参数、prompt 和 verifier 阈值的试验也应计入搜索预算。

6.8 分布外和对抗测试

OOD 不是简单换一个 seed，而是改变生成机制：

- 训练/搜索用均匀，审计用 Zipf 或 burst；
- 小规模搜索，大规模评估；
- 随机图换成高聚类、链、星或近完全图；
- 预测准确分布换成系统性漏掉 heavy hitter；
- 数值范围、稀疏度、符号或边界条件变化；
- 变量重命名、语法等价改写或不同 Lean 定义实现。

对抗测试应针对算法假设，而不是追求看起来复杂。一个两元素反例往往比百万规模随机输入更能说明漏洞。

6.9 Property-based testing 与 metamorphic relation

没有易得 oracle 时，可检查输入变换前后的关系。例如：

- 图顶点重命名不应改变目标值；
- 把流拆分后合并 sketch 应与整流处理一致；
- 矩阵同时做合法基变换后代数恒等式仍成立；
- 列表置换对声称 permutation-invariant 的函数不应改变结果；
- 给调度实例所有处理时间乘同一常数，目标值应相应缩放。

Metamorphic tests 仍是测试证据，但能发现只记忆表示细节的候选。

6.10 反例最小化协议

定义复杂度度量 $s(x)$ ，例如节点数、流长度、最大整数或 AST 节点数。对失败输入反复尝试变换 T_j ，只接受满足

$$\text{fails}(T_j(x)) \text{ 且 } s(T_j(x)) < s(x)$$

的变换，直到局部不可再缩。

最小反例报告应说明“相对于哪些缩小操作局部最小”，不要声称全局最小，除非做了完整枚举证明。

6.11 失败分类表

类别	例子	下一步
表示失败	语法错、非法状态、重复编码	收紧 grammar / 规范化
正确性失败	反例、proof goal 未关闭	缩小反例、补不变量
验证器失败	oracle 不一致、timeout 当 pass	修 verifier，重审历史候选
泛化失败	validation 高、OOD 崩溃	降低反馈、扩分布、正则约束
资源失败	内存/时间超过预算	重新计费、简化候选
研究失败	与已知方法等价、无显著收益	文献对照、缩小或终止 claim

Verifier bug 是最高优先级：修复后所有依赖旧 verifier 的结果都需要重跑或标记失效。

6.12 可复现运行清单

每次运行保存：Git commit、环境 lock、硬件/软件版本、完整配置、所有 seed、数据生成参数、候选与 verifier hash、原始逐实例结果、聚合脚本、stderr/timeout、wall-clock 和停止原因。

图表应由原始表脚本生成；不要手工修改最终数字。结果目录可包含生成物但应与公开仓库策略分开，避免把大数据、凭据或第三方受限材料推上 GitHub。

结论边界

“统计显著”不等于效应有研究价值，“测试全部通过”不等于一般正确，“Lean 编译”不等于自然语言语义忠实。每个结论只能上升到其最强证据允许的等级。

本章验收

1. 写出一个 confirmatory claim 和两个 exploratory 问题；
2. 区分 data、algorithm、model 和 split seed；

3. 为所有方法建立同口径预算表；
4. 解释零失败、Hoeffding 样本界和固定候选假设；
5. 设计一个 OOD 分布和两个 metamorphic relations；
6. 为一个反例定义复杂度与三个缩小操作；
7. 建立 verifier bug 后的历史结果失效规则。

第七章 两周项目执行与可复现交付

7.1 先用决策表只选一个项目

条件	项目 A	项目 B	项目 C
最接近已有 Sketch 背景	强	中	弱
最快得到误差曲线	强	中	弱
最能练 AI4TCS 闭环	中	强	中
最能练 kernel 证据	弱	中	强
主要风险	预测接口含糊	verifier 过拟合	Lean/概率接口过大
默认选择	首选	次选	Lean 基础稳定后

若当前目标是连接信息论、数据流和已有阅读，选 A；若想理解算法发现系统，选 B；若已能独立写 10 个 Lean theorem，选 C。不要为了“完整”同时启动三套代码。

7.2 第 0 天：冻结项目契约

用一页写清：

1. 一句话问题和不声称的结论；
2. object / representation / generator / verifier；
3. 一个主 baseline 和一个简单消融；
4. 主指标、预算和成功阈值；
5. development/validation/hidden/OOD 的生成方式；
6. 两周停止条件；
7. 最终要留下的文件。

契约允许版本化修改，但每次修改都要记录原因，不能无痕移动目标。

7.3 十个工作日节奏

天	核心任务	当日可检查产出
1	定义对象、假设和主 claim	Problem.md v1
2	实现 exact/oracle 与数据生成	三个手工例 + 一个边界例
3	跑通经典/简单 baseline	固定 schema 原始结果
4	写 verifier 与失败分类	一个故意错误候选被拒绝
5	冻结第一周配置	baseline 表 + 已知风险
6	实现唯一增强/生成策略	最小可运行方法
7	加入 validation 与查询预算	完整搜索/实验轨迹
8	hidden/OOD 与反例缩小	至少一个失败案例
9	最终审计、消融和统计	summary.csv + 证据登记
10	写结论边界与下一步	4-6 页研究笔记

Lean 项目可把第 2-5 天替换为有限枚举、定义、结构 lemma 和主 theorem，但仍保留 hidden 变体与失败记录。

7.4 目录模板

```
project/
  README.md
  Problem.md
  config/
    baseline.yaml
    main.yaml
  src/
  scripts/
  tests/
    development/
    hidden_generator.py
  notes/
    method.md
    failure_analysis.md
    evidence_ledger.md
  results/
    raw.csv
    summary.csv
  environment/
    requirements.txt
```

配置、代码、结果和叙述分开。raw.csv 每行对应一个实验单位；summary.csv 由脚本聚合。Lean 项目用 Lake 文件替换 Python 环境，并增加 ProofStatus.md。

7.5 三个项目的阶段闸门

7.5.1 项目 A

进入增强算法前, exact counter 与经典 sketch 必须在手工流上吻合预期; 空间计费固定; 主误差指标冻结。进入最终实验前, 必须存在可控 prediction error 生成器和一个错误预测反例。

7.5.2 项目 B

进入搜索前, verifier 必须拒绝至少五类故意错误候选; oracle 与候选实现独立; hidden 查询策略冻结。进入最终审计前, 候选等价去重和 timeout/unknown 处理已实现。

7.5.3 项目 C

进入主 theorem 前, 三列表和有限 Python 枚举完成; 进入最终交付前, lake build 干净通过、无 sorry/新增目标公理, 并保存一次最小失败修复记录。

7.6 证据登记表模板

Claim	Evidence	Remaining gap
固定分布下 p95 误差下降	20 个独立数据 seed 的 paired 结果	未证明 worst-case; 只覆盖给定参数
候选对 $n \leq 8$ 正确	全枚举与独立 oracle	任意 n 尚无证明
Lean statement 可证明	固定 commit 的无占位 build	自然语言对齐仍由人工审查

每个摘要结论都必须在表中有一行; 没有证据行的句子不能进入结论段。

7.7 研究笔记结构

4–6 页 Markdown 或 PDF 足够:

1. 问题、动机与最小 claim;
2. 对象、表示、baseline 与 verifier;
3. 方法或候选空间;
4. 数据、预算、指标和复现命令;
5. 主结果与不确定性;

6. 一个成功例与一个失败/反例；
7. 证据边界和不声称内容；
8. 下一步 go / revise / stop 判断。

不要用长背景综述淹没真正实验。相关工作只需要足以判断 baseline 和新意。

7.8 Go / revise / stop

项目结束必须做决定：

go 主效应跨 seed/分布稳定，verifier 可信，下一步可尝试理论化或扩大规模；

revise 有局部信号但依赖某个假设，需要更换 prediction interface、表示或 verifier；

stop 固定预算无收益、简单 baseline 同样好、核心 claim 有小反例，或实现成本超出价值。

停止结论同样是交付，只要反例、配置和原因可审计。

7.9 从两周项目到研究问题

项目 A 若成功，可问 prediction error 能否进入一般误差界、fallback 是否有最坏情况保证、mergeability 与 side information 是否兼容。项目 B 若成功，可研究反馈精度与泛化、反例引导 search policy、等价类表示或形式审计成本。项目 C 若成功，可扩展 theorem 族、autoformalization 语义评测、proof repair 数据或有限概率库接口。

升级前先补经典 baseline 与相关工作。实验曲线不能直接跳到新 theorem，固定 theorem 也不能直接跳到通用 agent。

7.10 发布前安全与仓库审计

公开仓库检查：无本机绝对路径、token/凭据、私有数据、论文 PDF、巨大运行产物或第三方受限代码；README 包含许可与复现边界；生成 PDF 保留对应源码；中间编译文件和视觉检查图片不入库。

结果若含模型 API 调用，记录模型标识、日期、参数和预算，但不提交密钥。若候选代码不可信，使用隔离环境与资源限制，不让它访问主科研目录。

可执行协议

最终验收不是“代码能跑”，而是：从干净环境可复现主结果；错误候选会被 verifier 拒绝；至少一个失败被解释；证据登记表与摘要 claim 一一对应；仓库只含允许公开的源码、配置、小型结果和文档。

本章验收

1. 用决策表选定且只选一个两周项目；
2. 完成第 0 天项目契约；
3. 为自己的项目定义三个不可跳过的阶段闸门；
4. 建立 raw/summary 分离的结果 schema；
5. 给每个 claim 填 evidence 与 remaining gap；
6. 在结项时明确选择 go、revise 或 stop。

第八章 综合习题与参考答案

8.1 习题

- X1. 分别给出一个 AI4Math、AI4TCS 和 TCS4AI 问题，并解释边界。
- X2. 对“语言模型生成更短的排序程序”填写 object、representation、generator、verifier、guarantee、generalization 六字段。
- X3. 某 verifier 只在 200 个随机输入上测试候选。它的 soundness 与 completeness 能否成立？应该怎样准确表述？
- X4. 每轮 verifier 返回 0–999 的整数分数，搜索 100 轮。忽略其他侧信道时，transcript 最多约含多少 bit？为什么这不是泛化保证？
- X5. 解释为什么用 Count-Min 直接 sketch $f - p$ 可能破坏单侧误差证明，并给出两种修复方向。
- X6. 对归一化频率预测，比较 L_1 、KL 和 heavy-hitter recall；各自在哪种失败上敏感？
- X7. 预测 heavy-hitter exact counter + 完整 backup sketch 为什么容易展示“永不变差”？固定总内存后应怎样公平比较？
- X8. 两个流分片使用不同 heavy set 和不同自适应宽度。说明它们的学习增强 sketch 为什么未必可直接合并。
- X9. 为一个小图遍历规则候选设计五层 verifier，并写出每层返回的失败信息。
- X10. Freivalds 单次错误接受概率至多 $1/2$ 。要使其不超过 10^{-6} ，至少独立重复多少次？
- X11. 固定候选在 1000 个独立测试上零失败。用 rule of three 给出约 95% 失败率上界，并说明不能推出什么。
- X12. 一个搜索器每天查看 hidden 分数并修改候选。为什么这份 hidden 已不再是最终独立测试？怎样修复协议？
- X13. 把“有限事件族 union bound”拆成自然语言、数学式和 Lean 设计，并列出至少四个形式化假设。
- X14. 给出一个 kernel-checked theorem 的证据登记行：claim、evidence、remaining gap。

X15. 固定尺寸 8×8 矩阵乘法候选在某 GPU 上更快。写出可以声称和不能声称的结论。

X16. 从 A/B/C 中选一个项目，写第 0 天契约、三个阶段闸门和 go/revise/stop 条件。

8.2 参考答案与提示

X1. AI4Math: 把自然语言组合恒等式翻译为 Lean 并证明; AI4TCS: 搜索一个小型调度启发式并用精确 oracle/hidden tests 验证; TCS4AI: 分析一个适应性搜索器从 verifier 反馈中获得多少信息以及何时泛化。真实项目可交叉, 但当前研究问题必须标注主要方向。

X2. Object 是排序程序; representation 可为受限 AST/DSL; generator 是语言模型; verifier 先检查所有测试/形式规范的功能正确性, 再计算代码长度或操作数; guarantee 若只有有限穷举就只覆盖该范围; generalization 要测试新长度、数据分布、元素类型和等价语法。还需说明“更短”按 token、AST 节点还是机器指令计。

X3. 对任意输入性质而言, 200 个测试通常不能给 soundness: 错误候选可能在未测输入失败。对“通过这 200 个测试”这一较弱性质, verifier 可精确 sound/complete。准确表述是固定候选在指定分布、seed 和预算下经验通过率; 若有独立同分布假设, 可给统计界, 但仍不是 worst-case proof。

X4. 每个分数有 1000 种值, 信息最多 $\log_2 1000 \approx 9.97$ bit, 100 轮 transcript 至多约 997 bit。实际互信息可能更低或受其他反馈影响。这个上界只约束反馈容量, 不自动说明候选分布、搜索算法稳定性或 hidden 数据上的泛化。

X5. Residual $r_i = f_i - p_i$ 可为负, 碰撞噪声不再全非负, Count-Min 的“估计永不低于真值”和 Markov 分析失效。可改用 Count Sketch 等支持有符号更新的估计器, 或分别 sketch 正/负 residual 并重新推导组合误差; 也可限制 prediction 使 residual 非负, 但会改变接口。

X6. L_1 衡量总质量偏差, 对许多小误差可累积; KL 对真实高概率但预测接近零的 key 极敏感, 并有支持集/方向问题; heavy-hitter recall 直接惩罚漏掉大 key, 却忽略尾部概率拟合。选择应匹配算法如何使用预测, 且主指标应在看结果前冻结。

X7. 完整 backup 使用了经典 baseline 的全部内存, 再加 exact counters, 当然容易不劣。固定总内存时, 应把 exact counter、key 标识、预测参数、hash seed 和计数器统一计费, 然后缩小 backup 或 baseline 到相同预算; 重新比较误差、更新成本和错误预测退化。

X8. 经典线性 sketch 合并要求相同维度、哈希和线性状态语义。不同分片若使用不同 heavy set/宽度, 状态坐标和 exact/sketch 分工不一致, 相加不等于对并流运行同一算法。可冻结全局预测配置, 或设计携带可协调 metadata 的合并协议并证明等价性。

X9. 接口层检查图格式、返回序列和超时; 手工层检查空图、单点、链和断连图; 小规模层枚举所有小图并由独立 BFS/DFS oracle 验证; hidden/OOD 层更换图族、规模与重命名; 最终层证明访问不变量/覆盖性质并分析 $O(|V| + |E|)$ 是否成立。每层返回可重放输入、预期/实际性质、错误类和 verifier version。

X10. 需 $2^{-k} \leq 10^{-6}$, 即 $k \geq \log_2 10^6 \approx 19.93$, 所以至少 20 次独立重复。该计算依赖域上正确实现、错误候选固定和随机向量独立。

X11. 约 95% 上界为 $3/1000 = 0.003$ 。不能推出真实失败率为 0, 不能推出 worst-case 正确, 也不能在候选根据这 1000 个测试反复选择后直接使用固定候选解释。

X12. 每天的分数进入了搜索历史, 因此搜索器适应了 hidden; 它实际是 validation。应冻结当前候选, 在从未查询的新 final/OOD 集上一次复验, 并为以后限制 hidden 查询次数、反馈精度和里程碑; 完整记录已经泄漏的轮次。

X13. 自然语言: 同一有限均匀空间中有限事件族并集概率不超过概率和。数学式为

$$\Pr\left[\bigcup_{i \in I} E_i\right] \leq \sum_{i \in I} \Pr[E_i].$$

Lean 设计需选择有限非空样本空间、有限索引类型/Finset、可判定事件、同一概率定义、事件并集表示和有序数值域; 若使用一般概率库, 还需相应可测性条件。

X14. Claim: 固定 toolchain 下 theorem T 可证明。Evidence: 公开 commit 上无 sorry/新增目标公理的 lake build, 并记录 `#print axioms T`。Remaining gap: 自然语言 statement 对齐由人工审查; 外部 Python 实现、概率模型或复杂度若未进入 T , 不在证明范围。

X15. 可以声称: 在指定 GPU、实现、数据类型、batch 和计时协议上, 固定 8×8 候选的实测延迟/吞吐优于 baseline。不能仅据此声称矩阵乘法指数下降、任意尺寸更快、乘法次数理论最优或在其他硬件保持优势。还需正确性、数值稳定性和端到端成本审计。

X16. 答案应具体到一个项目。以 A 为例: 契约固定 insertion-only、Count-Min baseline、上一窗口 heavy-set 预测、固定总内存、p95 绝对误差和 drift/adversarial miss; 闸门为 baseline 对拍、prediction error 生成器完成、错误预测反例已保存; go 要求跨 seed/分布

稳定收益且退化达标, revise 表示只在某接口有效, stop 表示简单 baseline 同样好或固定预算无收益。

参考文献

- [1] Alhussein Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 2022.
- [2] Bernardino Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 2024.
- [3] Trieu H. Trinh et al. Solving olympiad geometry without human demonstrations. *Nature*, 2024.
- [4] Kaiyu Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. *NeurIPS*, 2023.
- [5] Kunhao Zheng et al. miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. *ICLR*, 2022.
- [6] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. In *Beyond the Worst-Case Analysis of Algorithms*, 2020.
- [7] Graham Cormode and Ke Yi. *Small Summaries for Big Data*. Cambridge University Press, 2020.